

Compiler Design Lab Report 4

YACC-Based Arithmetic Expression Syntax Analyzer with Symbol Table Management

Abdullah Al Jubaer Gem

ID: 210041226

Section: 2B

June 20, 2026

1 Objective

Extend the YACC-based arithmetic expression parser from Lab 3 with variable support and a dynamic symbol table. The system recognizes INTEGER, DOUBLE, and IDENTIFIER tokens, handles assignment statements, and persists variable state across executions using a linked list symbol table saved to a file.

2 Expression Grammar

The grammar extends the base arithmetic grammar with assignment statements, identifier lookup, and mathematical function support. Operator precedence is encoded structurally — function application and exponentiation bind tightest, followed by multiplication/division, then addition/subtraction.

$$\begin{aligned}\text{statement} &\rightarrow \text{ID} = \text{expr} \mid \text{expr} \\ \text{expr} &\rightarrow \text{expr} + \text{term} \mid \text{expr} - \text{term} \mid \text{term} \\ \text{term} &\rightarrow \text{term} * \text{power} \mid \text{term} / \text{power} \mid \text{power} \\ \text{power} &\rightarrow \text{factor}^{\text{power}} \mid \text{factor} \\ \text{factor} &\rightarrow \sin(\text{expr}) \mid \cos(\text{expr}) \mid \tan(\text{expr}) \mid \sqrt{\text{expr}} \mid \log(\text{expr}) \\ &\mid (\text{expr}) \mid \text{INTEGER} \mid \text{DOUBLE} \mid \text{ID}\end{aligned}$$

3 Background Theory

A syntax analyzer (parser) verifies whether a sequence of tokens follows grammatical rules. In compiler design, Flex performs lexical analysis by converting input into tokens, and YACC constructs a parser based on grammar rules.

A symbol table is introduced in this lab to store identifiers and their values during execution. It is implemented as a linked list to allow dynamic memory allocation. Each entry stores the variable name, its current value, and its type (INTEGER or DOUBLE). File I/O persists the symbol table between executions via `symbols.txt`.

4 Implementation

4.1 FLEX Lexer

```
1 %{
2 #include "y.tab.h"
3 #include <stdlib.h>
4 #include <string.h>
```

```

5  %}
6
7  %%
8
9  [0-9]+\.[0-9]+([eE][+-]?[0-9]+)? {
10     yylval.dval = atof(yytext);
11     return DOUBLE;
12 }
13
14 [0-9]+ {
15     yylval.dval = atof(yytext);
16     return INTEGER;
17 }
18
19 [a-zA-Z_][a-zA-Z0-9_]* {
20     yylval.sval = strdup(yytext);
21     return IDENTIFIER;
22 }
23
24 [ \t\r]+      ;
25
26 \n            return '\n';
27 ";"          return ';';
28
29 "+"          return '+';
30 "-"          return '-';
31 "*"          return '*';
32 "/"          return '/';
33 "^"          return '^';
34 "("          return '(';
35 ")"          return ')';
36 "="          return '=';
37
38 .            return yytext[0];
39
40 %%
41
42 int yywrap()
43 {
44     return 1;
45 }

```

Listing 1: FLEX Lexer (lexer.l)

4.2 YACC Parser

```

1  %{
2  #include <stdio.h>
3  #include <stdlib.h>
4  #include <string.h>
5  #include <math.h>
6
7  extern FILE *yyin;
8  extern char *yytext;
9
10 FILE *out;
11
12 int yylex(void);
13 void yyerror(const char *s);
14
15 typedef struct Symbol
16 {
17     char name[50];
18     double value;
19     int type;
20     struct Symbol *next;
21 } Symbol;
22
23 Symbol *head = NULL;
24
25 Symbol* createSymbol(char *name, double value, int type)
26 {
27     Symbol *n = (Symbol*)malloc(sizeof(Symbol));

```

```

28     strcpy(n->name, name);
29     n->value = value;
30     n->type = type;
31     n->next = NULL;
32     return n;
33 }
34
35 Symbol* findSymbol(char *name)
36 {
37     Symbol *t = head;
38     while (t)
39     {
40         if (strcmp(t->name, name) == 0)
41             return t;
42         t = t->next;
43     }
44     return NULL;
45 }
46
47 void setSymbol(char *name, double value)
48 {
49     Symbol *s = findSymbol(name);
50     int type = (floor(value) == value) ? 0 : 1;
51
52     if (!s)
53     {
54         Symbol *n = createSymbol(name, value, type);
55         n->next = head;
56         head = n;
57     }
58     else
59     {
60         s->value = value;
61         s->type = type;
62     }
63 }
64
65 double getSymbolValue(char *name, int *found)
66 {
67     Symbol *s = findSymbol(name);
68     if (!s)
69     {
70         *found = 0;
71         return 0;
72     }
73     *found = 1;
74     return s->value;
75 }
76
77 void saveSymbols()
78 {
79     FILE *fp = fopen("symbols.txt", "w");
80     Symbol *t = head;
81     while (t)
82     {
83         fprintf(fp, "%s %lf %d\n", t->name, t->value, t->type);
84         t = t->next;
85     }
86     fclose(fp);
87 }
88
89 void loadSymbols()
90 {
91     FILE *fp = fopen("symbols.txt", "r");
92     if (!fp) return;
93
94     char name[50];
95     double value;
96     int type;
97
98     while (fscanf(fp, "%s %lf %d", name, &value, &type) == 3)
99     {
100         Symbol *n = createSymbol(name, value, type);

```

```

101     n->next = head;
102     head = n;
103 }
104
105     fclose(fp);
106 }
107 %}
108
109 %union {
110     double dval;
111     char *sval;
112 }
113
114 %token <dval> INTEGER DOUBLE
115 %token <sval> IDENTIFIER
116
117 %type <dval> expr term factor primary
118
119 %left '+' '-'
120 %left '*' '/'
121 %right '^'
122 %right UMINUS
123
124 %%
125
126 input:
127     input line
128     |
129     ;
130
131 line:
132     '\n'
133     | statement '\n'
134     | statement ';'
135     | statement
136     ;
137
138 statement:
139     IDENTIFIER '=' expr
140     {
141         setSymbol($1, $3);
142         fprintf(out, "%s = %g\n", $1, $3);
143     }
144     | expr
145     {
146         fprintf(out, "Result = %g\n", $1);
147     }
148     ;
149
150 expr:
151     expr '+' term { $$ = $1 + $3; }
152     | expr '-' term { $$ = $1 - $3; }
153     | term { $$ = $1; }
154     ;
155
156 term:
157     term '*' factor { $$ = $1 * $3; }
158     | term '/' factor
159     {
160         if ($3 == 0)
161         {
162             fprintf(out, "Division by zero\n");
163             $$ = 0;
164         }
165         else $$ = $1 / $3;
166     }
167     | factor { $$ = $1; }
168     ;
169
170 factor:
171     primary '^' factor { $$ = pow($1, $3); }
172     | primary { $$ = $1; }
173     ;

```

```

174
175 primary:
176     '(' expr ')' { $$ = $2; }
177     | INTEGER { $$ = $1; }
178     | DOUBLE { $$ = $1; }
179     | IDENTIFIER
180     {
181         int f;
182         $$ = getSymbolValue($1, &f);
183         if (!f)
184         {
185             fprintf(out, "Warning: %s not defined\n", $1);
186             $$ = 0;
187         }
188     }
189     ;
190
191 %%
192
193 void yyerror(const char *s)
194 {
195     fprintf(out, "Syntax Error: %s near %s\n", s, yytext);
196 }
197
198 int main()
199 {
200     FILE *fp = fopen("input.txt", "r");
201     out = fopen("output.txt", "w");
202
203     if (!fp || !out)
204         return 1;
205
206     yyin = fp;
207
208     loadSymbols();
209     yyparse();
210     saveSymbols();
211
212     fclose(fp);
213     fclose(out);
214     return 0;
215 }

```

Listing 2: YACC Parser (parser.y)

5 Compilation Steps

```

1 yacc -d parser.y
2 flex lexer.l
3 gcc lex.yy.c y.tab.c -o parser -lm
4 ./parser

```

Listing 3: Build and Run

6 Sample Input and Output

6.1 Input (input.txt)

```

1 marks1 = 20.5
2 marks2 = 10
3 bonus = 5
4 total = marks1 + marks2 + bonus
5 avg = total / 3
6 result1 = avg * 2
7 result2 = result1 + 10
8 finalScore = result2 ^ 2
9
10 a = 100
11 b = a + 50
12 c = b / 2

```

```
13
14 undefinedTest = x + 5
```

Listing 4: Sample Input

6.2 Output (output.txt)

```
1 marks1 = 20.5
2 marks2 = 10
3 bonus = 5
4 total = 35.5
5 avg = 11.8333
6 result1 = 23.6667
7 result2 = 33.6667
8 finalScore = 1133.44
9 a = 100
10 b = 150
11 c = 75
12 Warning: x not defined
13 undefinedTest = 5
```

Listing 5: Generated Output

6.3 Symbol Table (symbols.txt)

```
1 undefinedTest 5.000000 0
2 c 75.000000 0
3 b 150.000000 0
4 a 100.000000 0
5 finalScore 1133.444444 1
6 result2 33.666667 1
7 result1 23.666667 1
8 avg 11.833333 1
9 total 35.500000 1
10 bonus 5.000000 0
11 marks2 10.000000 0
12 marks1 20.500000 1
```

Listing 6: Symbol Table File

7 Conclusion

The extended YACC-based parser successfully handles arithmetic expression evaluation, assignment statements, and identifier lookup through a dynamic symbol table. Flex tokenizes INTEGER, DOUBLE, and IDENTIFIER values, which YACC processes according to the defined grammar rules. The linked list implementation enables efficient runtime insertion and retrieval of variables, while file I/O ensures symbol table state persists across executions. Undefined variable references are caught and reported as warnings without halting the parser.